

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1**



ANDROID BASICS WITH COMPOSE

Oleh:

Noor Muhammad Akmal Sulaiman NIM. 2410817210007

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basics with Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Noor Muhammad Akmal Sulaiman
NIM : 2410817210007

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom., M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	4
DAFTAR TABEL.....	5
SOAL 1	6
A. Source Code	9
B. Output Program.....	15
C. Pembahasan.....	16
D. Essay	20
E. Daftar Pustaka	25
TAUTAN GIT.....	26

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi	6
Gambar 2. Tampilan Dadu Setelah Di-Roll.....	7
Gambar 3. Tampilan Roll Dadu Double	8
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose.....	15
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML	16

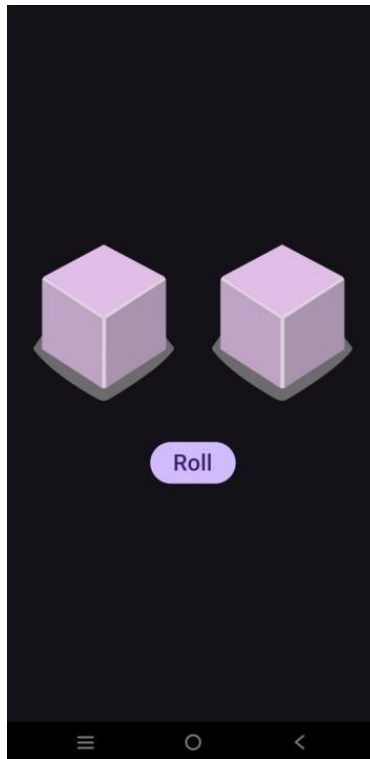
DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt Compose	9
Tabel 2. Source Code MainActivity.kt XML.....	12
Tabel 3. Source Code activity_main.xml XML	14

SOAL 1

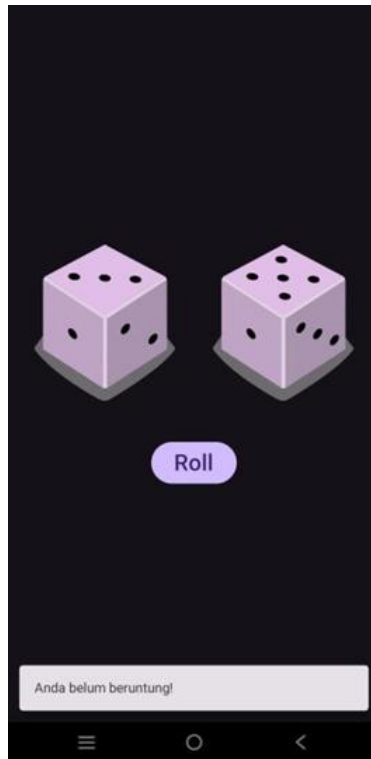
Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



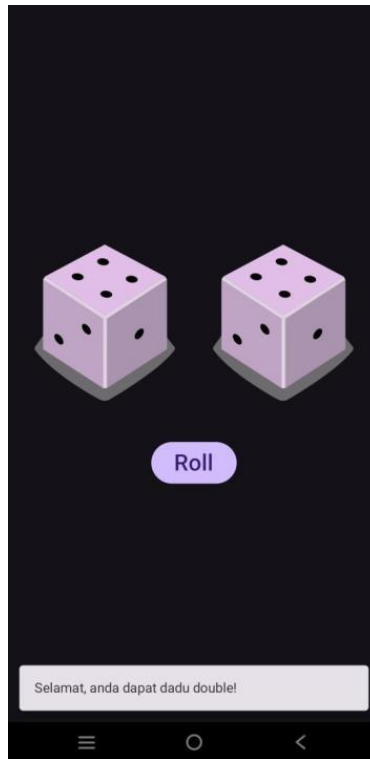
Gambar 1. Tampilan Awal Aplikasi

2. Setelah user menekan tombol “Roll” maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan “Anda belum beruntung!” seperti yang dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah Di-Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat, anda dapat dadu double!” seperti yang dapat dilihat pada Gambar 3.



Gambar 3. Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder Modul 1 dalam bentuk Project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat di-download pada link berikut:
https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp=sharing

Additional:

ESSAY

1. Riset dan cite artikel/jurnal/buku mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatlah opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan.
**
2. Apa yang membedakan OOP pada kotlin dengan OOP pada java? *
3. Sebutkan, dan jelaskan mengenai SOLID Principles *

4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

** = 2 sumber

* = 1 sumber

Gunakanlah sumber primary artikel/jurnal/buku, web hanya sebagai penunjang opini/argumen.

A. Source Code

MainActivity.kt (Jetpack Compose)

Tabel 1. Source Code MainActivity.kt Compose

```
1 package io.github.orizynpx.dicerollercompose
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.enableEdgeToEdge
7 import androidx.compose.foundation.Image
8 import androidx.compose.foundation.layout.fillMaxSize
9 import
10     androidx.compose.foundation.layout.wrapContentSize
11 import androidx.compose.material3.Text
12 import androidx.compose.runtime.Composable
13 import androidx.compose.ui.Alignment
14 import androidx.compose.ui.Modifier
15 import androidx.compose.ui.tooling.preview.Preview
16 import
17     io.github.orizynpx.dicerollercompose.ui.theme.DiceRoller
18     ComposeTheme
19 import androidx.compose.foundation.layout.Column
20 import androidx.compose.foundation.layout.Row
21 import androidx.compose.material3.Button
22 import androidx.compose.ui.res.stringResource
23 import androidx.compose.foundation.layout.Spacer
24 import androidx.compose.foundation.layout.height
25 import androidx.compose.foundation.layout.padding
26 import androidx.compose.material3.Scaffold
27 import androidx.compose.material3.SnackbarHost
28 import androidx.compose.material3.SnackbarHostState
29 import androidx.compose.runtime.MutableState
30 import androidx.compose.runtime.remember
31 import androidx.compose.ui.res.painterResource
32 import androidx.compose.ui.unit.dp
33 import androidx.compose.runtime.mutableStateOf
34 import androidx.compose.runtime.rememberCoroutineScope
35 import kotlinx.coroutines.launch
```

```

34 class MainActivity : ComponentActivity() {
35     override fun onCreate(savedInstanceState: Bundle?) {
36         super.onCreate(savedInstanceState)
37         enableEdgeToEdge()
38         setContent {
39             DiceRollerComposeTheme {
40                 DiceRollerApp()
41             }
42         }
43     }
44 }
45
46 @Preview
47 @Composable
48 fun DiceRollerApp() {
49     DiceWithButtonAndImage(modifier = Modifier
50         .fillMaxSize()
51         .wrapContentSize(Alignment.Center)
52     )
53 }
54
55 @Composable
56 fun DiceWithButtonAndImage(modifier: Modifier =
57     Modifier) {
58     val result1: MutableState<Int> = remember {
59         mutableStateOf(0) }
60     val imageResource1: Int = when (result1.value) {
61         1 -> R.drawable.dice_1
62         2 -> R.drawable.dice_2
63         3 -> R.drawable.dice_3
64         4 -> R.drawable.dice_4
65         5 -> R.drawable.dice_5
66         6 -> R.drawable.dice_6
67         else -> R.drawable.dice_0
68     }
69     val result2: MutableState<Int> = remember {
70         mutableStateOf(0) }
71     val imageResource2: Int = when (result2.value) {
72         1 -> R.drawable.dice_1
73         2 -> R.drawable.dice_2
74         3 -> R.drawable.dice_3
75         4 -> R.drawable.dice_4
76         5 -> R.drawable.dice_5
77         6 -> R.drawable.dice_6
78         else -> R.drawable.dice_0
79     }

```

```

79     val diceWinMessage =
stringResource(R.string.dice_win_msg)
80     val diceLoseMessage =
stringResource(R.string.dice_lose_msg)
81
82     val scope = rememberCoroutineScope()
83     val snackbarHostState = remember {
SnackbarHostState() }
84
85     Scaffold(
86         snackbarHost = {
87             SnackbarHost(snackbarHostState)
88         }
89     ) { contentPadding ->
90         Column(
91             modifier = modifier.padding(contentPadding),
92             horizontalAlignment =
Alignment.CenterHorizontally
93         ) {
94             Row {
95                 Image(
96                     painter =
painterResource(imageResource1),
97                     contentDescription =
result1.toString()
98                 )
99                 Image(
100                     painter =
painterResource(imageResource2),
101                     contentDescription =
result2.toString()
102                 )
103             }
104
105             Spacer(
106                 modifier = Modifier.height(16.dp)
107             )
108
109             Button(
110                 onClick = {
111                     result1.value = (1..6).random()
112                     result2.value = (1..6).random()
113
114                     scope.launch {
115                         snackbarHostState.showSnackbar(
116                             if (result1.value ==
result2.value) diceWinMessage else diceLoseMessage
117                         )
118                     }

```

119	}
120	) {
121	Text(stringResource(R.string.roll))
122	}
123	}
124	}
125	}

MainActivity.kt (XML)

Tabel 2. Source Code MainActivity.kt XML

1	package io.github.orizynpx.dicerollerxml
2	
3	import android.os.Bundle
4	import android.widget.Button
5	import android.widget.ImageView
6	import androidx.activity.enableEdgeToEdge
7	import androidx.appcompat.app.AppCompatActivity
8	import androidx.constraintlayout.widget.ConstraintLayout
9	import androidx.core.view.ViewCompat
10	import androidx.core.view.WindowInsetsCompat
11	import com.google.android.material.snackbar.Snackbar
12	
13	class MainActivity : AppCompatActivity() {
14	private lateinit var imgDice1: ImageView
15	private lateinit var imgDice2: ImageView
16	
17	override fun onCreate(savedInstanceState: Bundle?) {
18	super.onCreate(savedInstanceState)
19	enableEdgeToEdge()
20	setContentView(R.layout.activity_main)
21	
22	imgDice1 = findViewById(R.id.imgDice1)
23	imgDice1.setImageResource(R.drawable.dice_0)
24	
25	imgDice2 = findViewById(R.id.imgDice2)
26	imgDice2.setImageResource(R.drawable.dice_0)
27	
28	val btnRoll: Button = findViewById(R.id.btnRoll)
29	btnRoll.setOnClickListener { rollDice() }
30	
31	
	ViewCompat.setOnApplyWindowInsetsListener(findViewById(R
	.id.main)) { v, insets ->
32	val systemBars =
	insets.getInsets(WindowInsetsCompat.Type.systemBars())
33	v.setPadding(systemBars.left,
	systemBars.top, systemBars.right, systemBars.bottom)

```

34         insets
35     }
36 }
37
38 private fun rollDice() {
39     val result1: Int = (1..6).random()
40     val result2: Int = (1..6).random()
41
42     val imageResource1 = when (result1) {
43         1 -> R.drawable.dice_1
44         2 -> R.drawable.dice_2
45         3 -> R.drawable.dice_3
46         4 -> R.drawable.dice_4
47         5 -> R.drawable.dice_5
48         6 -> R.drawable.dice_6
49         else -> R.drawable.dice_0
50     }
51     imgDice1.setImageResource(imageResource1)
52     imgDice1.contentDescription = result1.toString()
53
54     val imageResource2 = when (result2) {
55         1 -> R.drawable.dice_1
56         2 -> R.drawable.dice_2
57         3 -> R.drawable.dice_3
58         4 -> R.drawable.dice_4
59         5 -> R.drawable.dice_5
60         6 -> R.drawable.dice_6
61         else -> R.drawable.dice_0
62     }
63     imgDice2.setImageResource(imageResource2)
64     imgDice2.contentDescription = result2.toString()
65
66     val layout: ConstraintLayout =
67         findViewById(R.id.main)
68         val diceMessage = if (result1 == result2)
69             getString(R.string.dice_win_msg) else
70             getString(R.string.dice_lose_msg)
71
72     val snackbar = Snackbar.make(layout,
73         diceMessage, Snackbar.LENGTH_SHORT)
74     snackbar.show()
75 }

```

activity_main.xml (XML)

Tabel 3. Source Code activity_main.xml XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/main"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".MainActivity">
9	
10	
11	<LinearLayout
12	android:layout_width="match_parent"
13	android:layout_height="match_parent"
14	android:gravity="center"
15	android:orientation="vertical">
16	
17	<LinearLayout
18	android:layout_width="wrap_content"
19	android:layout_height="wrap_content"
20	android:orientation="horizontal">
21	
22	<ImageView
23	android:id="@+id/imgDice1"
24	android:layout_width="wrap_content"
25	android:layout_height="wrap_content"
26	android:src="@drawable/dice_0" />
27	
28	<ImageView
29	android:id="@+id/imgDice2"
30	android:layout_width="wrap_content"
31	android:layout_height="wrap_content"
32	android:src="@drawable/dice_0" />
33	
34	</LinearLayout>
35	
36	<Space
37	android:layout_width="wrap_content"
38	android:layout_height="16dp" />
39	
40	<Button
41	android:id="@+id/btnRoll"
42	android:layout_width="wrap_content"
43	android:layout_height="wrap_content"
44	android:text="@string/roll" />

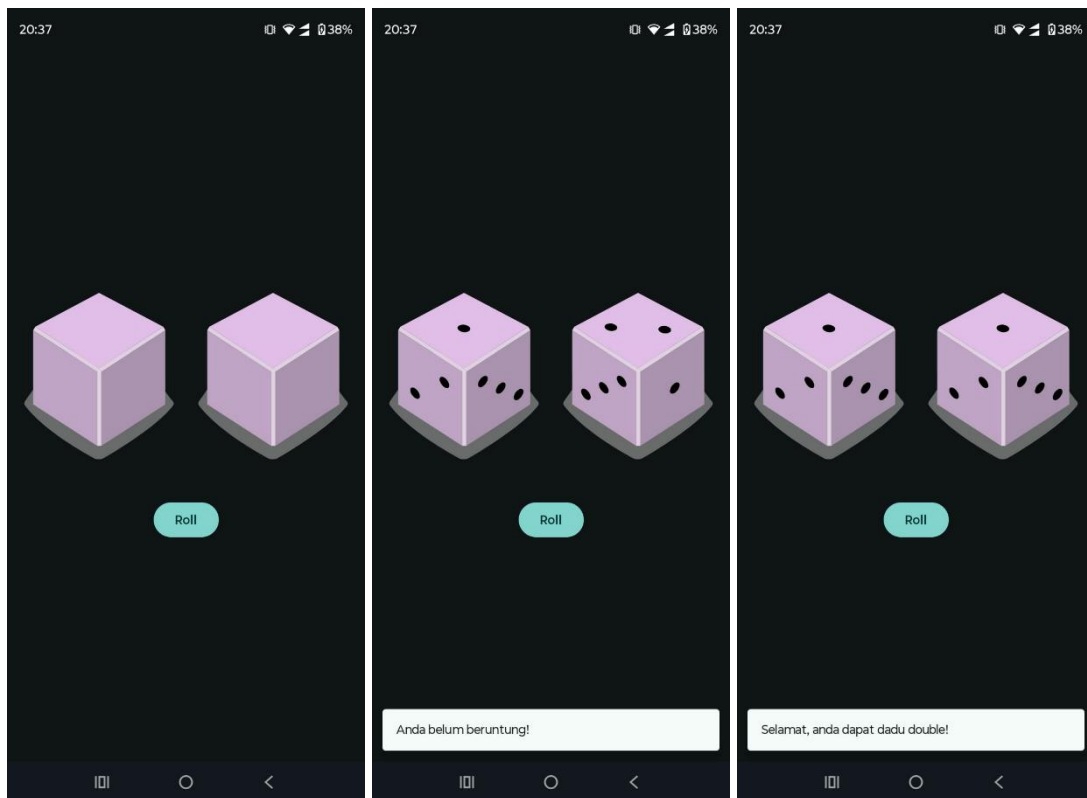
```

45
46     </LinearLayout>
47
48 </androidx.constraintlayout.widget.ConstraintLayout>

```

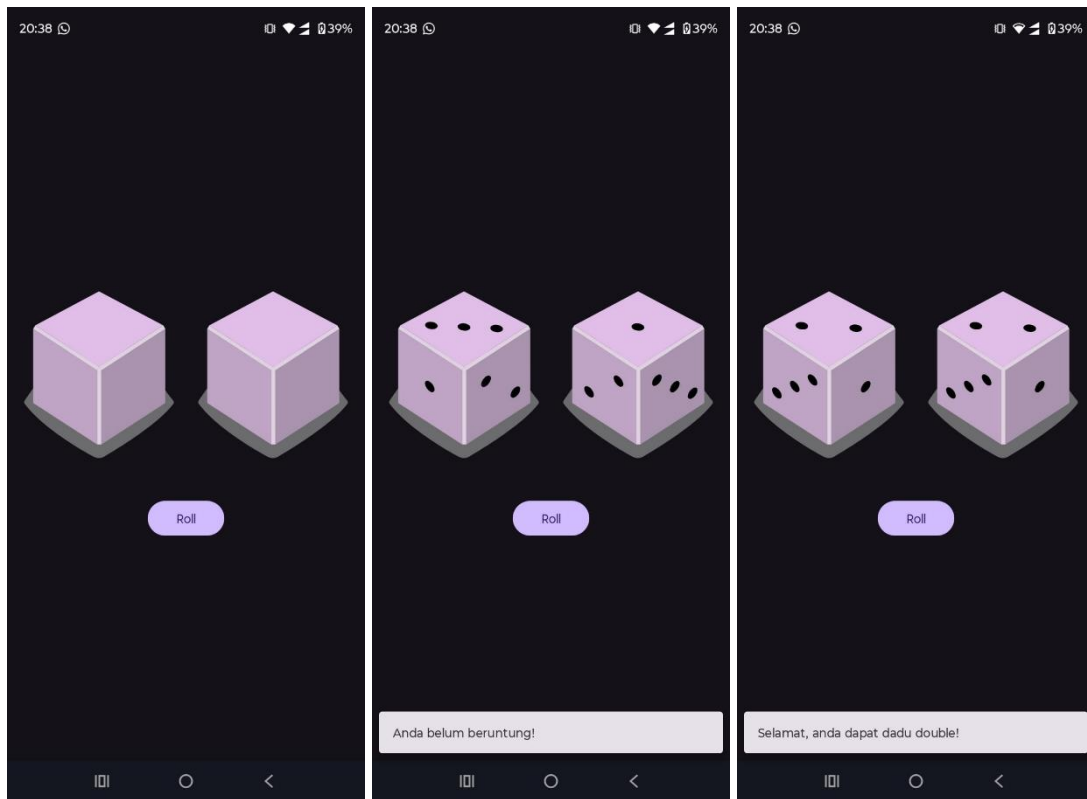
B. Output Program

Jetpack Compose



Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose

XML



Gambar 5. Screenshot Hasil Jawaban Soal 1 XML

C. Pembahasan

MainActivity.kt (Jetpack Compose)

Pada baris [1], terdapat pernyataan (*statement*) `package io.github.orizynpx.dicerollercompose` yang mendeklarasikan bahwa semua kode dalam *file* `MainActivity.kt` ini merupakan bagian dari paket (*package*) dengan nama tersebut, dan berfungsi sebagai pengenalan unik untuk aplikasi ini.

Pada baris [3] hingga [32], terdapat banyak pernyataan `import` yang digunakan untuk menyertakan berbagai kelas dan fungsi yang dibutuhkan dari *library* Android Jetpack Compose dan *library* standar Android, misalnya komponen UI seperti `Image`, `Button`, `Text`, dan `Scaffold`, serta fungsi-fungsi untuk tata letak (`Column`, `Row`, `Modifier`), manajemen *state* (`remember`, `mutableStateOf`), dan fungsionalitas lainnya seperti `CoroutineScope` untuk operasi asinkron.

Pada baris [34] hingga [44], terdapat kelas `MainActivity` yang meng-*inherit* kelas `ComponentActivity`. Metode `onCreate()` adalah titik masuk utama untuk sebuah *activity* Android. Di dalamnya, `setContent` digunakan untuk mendefinisikan UI aplikasi menggunakan Jetpack Compose. Seluruh UI dibungkus dalam tema aplikasi (`DiceRollerComposeTheme`), yang memanggil `composable` `DiceRollerApp` untuk membangun antarmuka.

Pada baris [46] hingga [53], terdapat fungsi `DiceRollerApp` yang merupakan sebuah *composable* dan berfungsi sebagai *container* utama untuk UI aplikasi. Anotasi `@Composable` menandakan bahwa ini adalah fungsi yang dapat membangun UI. Anotasi `@Preview` memungkinkan Android Studio untuk menampilkan pratinjau dari *composable* ini di panel desain. Fungsi ini memanggil *composable* `DiceWithButtonAndImage` dan memberikannya `Modifier` untuk menempatkan konten di tengah layar.

Pada baris [48] hingga [125], terdapat fungsi `DiceWithButtonAndImage` yang merupakan *composable* inti yang berisi semua logika dan elemen UI aplikasi. Di dalamnya, dua variabel *state* `result1` dan `result2` dibuat menggunakan `remember { mutableStateOf(0) }`. Ini adalah variabel yang nilainya dapat diobservasi oleh Compose. Setiap kali nilainya berubah, UI akan secara otomatis diperbarui (*recompose*). Terdapat dua blok `when` yang berfungsi untuk memilih gambar dadu yang sesuai (`R.drawable.dice_...`) berdasarkan nilai dari `result1` dan `result2`. Selain itu, fungsi ini juga mengelola `SnackbarHostState` untuk menampilkan pesan *snackbar* (*pop-up* singkat) dan `CoroutineScope` untuk meluncurkan operasi *snackbar* secara asinkron. Tampilan utama dibangun menggunakan `Scaffold`, yang menyediakan struktur dasar aplikasi. Di dalam `Scaffold`, sebuah `Column` digunakan untuk menyusun elemen-elemen secara vertikal. Di dalam `Column` tersebut, sebuah `Row` menampung dua komponen `Image` untuk menampilkan kedua dadu secara berdampingan. Sebuah `Spacer` memberikan jarak vertikal antara gambar dadu dan tombol. Terakhir, sebuah `Button` ditampilkan dengan teks “Roll”. Ketika tombol ini diklik (`onClick`), nilai `result1`

dan `result2` diacak dengan angka antara 1 hingga 6. Setelah itu, sebuah *coroutine* diluncurkan untuk menampilkan pesan *snackbar* yang memberitahu pengguna apakah mereka menang (jika nilai kedua dadu sama) atau kalah.

MainActivity.kt (XML)

Baris [1] hingga [11] hanya berisi pernyataan `package` dan `import` yang kurang lebih sama saja dengan versi Compose sebelumnya sehingga saya tidak akan mengulang penjelasannya.

Pada baris [13] hingga [72], terdapat kelas `MainActivity` yang mewarisi dari kelas `AppCompatActivity`, yang merupakan standar untuk *activity* yang mendukung fitur-fitur modern Android. Di dalam kelas ini, dua variabel `ImageView`, yaitu `imgDice1` dan `imgDice2`, dideklarasikan dengan `lateinit` yang menandakan bahwa nilainya akan diinisialisasi nanti.

Di dalam kelas tersebut, terdapat metode `onCreate()` sebagai titik masuk utama untuk sebuah *activity* Android. Di dalamnya, `setContentView(R.layout.activity_main)` digunakan untuk menghubungkan kelas ini dengan *file layout* XML-nya, yang mendefinisikan struktur visual antarmuka pengguna. Setelah itu, variabel `imgDice1` dan `imgDice2` diinisialisasi dengan menghubungkannya ke komponen `ImageView` yang sesuai di *file layout* menggunakan `findViewById()`. Gambar awal untuk kedua dadu diatur ke `dice_0`. Sebuah `Button` dengan id `btnRoll` juga diinisialisasi, dan `setOnClickListener` dipasang padanya. Blok kode di dalam *listener* ini, yaitu pemanggilan fungsi `rollDice()`, akan dieksekusi setiap kali tombol tersebut ditekan. Terakhir, `ViewCompat.setOnApplyWindowInsetsListener` digunakan untuk menyesuaikan *padding* dari *layout* utama agar tidak tertimpa oleh *system bar* (seperti *status bar*).

Fungsi `rollDice()` adalah fungsi privat yang berisi logika utama permainan. Ketika dipanggil, fungsi ini pertama-tama menghasilkan dua angka acak antara 1 hingga 6, yang disimpan dalam variabel `result1` dan `result2`. Terdapat dua blok `when` yang berfungsi untuk memilih sumber daya (*resource*) gambar

(`R.drawable.dice_...`) yang sesuai berdasarkan nilai `result1` dan `result2`. Gambar pada `imgDice1` dan `imgDice2` kemudian diperbarui menggunakan `setImageResource()`, dan `contentDescription` (teks untuk aksesibilitas) juga diatur sesuai dengan angka yang muncul. Setelah itu, fungsi ini menentukan pesan yang akan ditampilkan berdasarkan perbandingan `result1` dan `result2`. Terakhir, sebuah `Snackbar` dibuat untuk menampilkan pesan tersebut secara singkat di bagian bawah layar, dan `snackbar.show()` digunakan untuk menampilkannya kepada pengguna.

activity_main.xml (XML)

Deklarasi `<?xml version="1.0" encoding="utf-8"?>` di awal *file* adalah standar untuk semua *file* XML.

Tag akar (*root*) dari *layout* ini adalah `<androidx.constraintlayout.widget.ConstraintLayout>`, yang merupakan sebuah `ViewGroup` fleksibel yang memungkinkan penempatan elemen-elemen UI dengan mendefinisikan hubungan (*constraints*) antarelemen. Atribut `xmlns` digunakan untuk mendeklarasikan *namespace* yang dibutuhkan, `android:id="@+id/main"` memberikan identitas unik pada *layout* ini agar dapat direferensikan dari kode Kotlin, dan `android:layout_width` serta `android:layout_height` dengan nilai `match_parent` membuat *layout* ini mengisi seluruh layar.

Di dalam `ConstraintLayout`, terdapat sebuah `<LinearLayout>`. Atribut `android:gravity="center"` dan `android:orientation="vertical"` pada `LinearLayout` ini berfungsi untuk menempatkan semua elemen di dalamnya secara vertikal dan memosisikannya di tengah-tengah layar, baik secara horizontal maupun vertikal.

Di dalam `LinearLayout` vertikal tersebut, terdapat sebuah `<LinearLayout>` lain yang bersifat horizontal (`android:orientation="horizontal"`). *Layout* ini berfungsi sebagai wadah untuk dua buah dadu agar dapat ditampilkan berdampingan.

Di dalam `LinearLayout` horizontal, terdapat dua tag `<ImageView>`. Masing-masing `ImageView` ini memiliki `android:id` yang unik (`@+id/imgDice1` dan `@+id/imgDice2`) agar bisa diidentifikasi dan dimanipulasi dari kode Kotlin. Atribut `android:src="@drawable/dice_0"` mengatur gambar awal yang akan ditampilkan pada masing-masing `ImageView`, yaitu gambar `dice_0`.

Setelah `LinearLayout` horizontal, terdapat tag `<Space>`. Elemen ini digunakan untuk memberikan ruang atau jarak vertikal kosong sebesar `16dp` antara baris gambar dadu dan tombol di bawahnya, menciptakan tata letak yang lebih rapi.

Terakhir, di bawah `Space`, terdapat sebuah tag `<Button>`. Tombol ini memiliki `android:id` unik (`@+id/btnRoll`) dan menampilkan teks yang diambil dari sumber daya *string* (`@string/roll`). Tombol ini akan berfungsi sebagai pemicu untuk mengacak dadu saat ditekan oleh pengguna.

D. Essay

Soal 1. Riset dan cite artikel/jurnal/buku mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatlah opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan.

Saya melakukan riset singkat dengan membaca dua buah artikel yang membandingkan paradigma *imperative* dan *declarative*. Yang pertama adalah artikel dari Nurhadi dan Muslim (2025) dan yang kedua adalah artikel dari Fjodorovs dan Kodors (2021) [1], [2]. Nurhadi dan Muslim melakukan penelitian berupa *comparative analysis* untuk membandingkan dampak dari paradigma *declarative* dan *imperative* terhadap performa aplikasi Android menggunakan metode *benchmarking* [1]. Fjodorovs dan Kodors melakukan penelitian yang membandingkan performa dari *layout rendering* dari Jetpack Compose (*declarative*) dan XML (*imperative*) [2].

Nurhadi dan Muslim menemukan bahwa *imperative programming* menggunakan Kotlin dan XML meningkatkan kestabilan *UI rendering* dan keefisienan memori sebagaimana ditunjukkan oleh penggunaan RAM yang lebih rendah. Sementara itu,

declarative programming menggunakan Jetpack Compose meningkatkan keefisienan tenaga komputasi melalui penurunan penggunaan CPU rata-rata [1].

Fjodorovs dan Kodors menemukan bahwa Jetpack Compose meningkatkan lama waktu *rendering* sebesar 46% dibandingkan XML. Namun, ia mengatakan bahwa bisa saja hal tersebut disebabkan oleh ia yang menggunakan Jetpack Compose versi beta04 pada April 2021 [2]. Singkatnya, ketika mereka melakukan penelitian, Jetpack Compose terbilang cukup lambat dibandingkan XML dalam hal *UI rendering*.

Melihat penemuan tersebut, mudah untuk kita berpikir bahwa *imperative programming* (XML) lebih cocok digunakan untuk pengembangan UI pada aplikasi Android mengingat pada umumnya perangkat Android lebih sensitif akan hal-hal berbau performa. Namun, ada hal yang patut kita ingat selain performa perangkatnya sendiri, yaitu performa atau kinerja *developer* di balik layar.

Konon katanya *declarative programming* meningkatkan produktivitas *developer* karena teknologi dengan paradigma tersebut (misalnya Jetpack Compose atau bahkan bahasa Kotlin itu sendiri) jauh lebih minim *boilerplate* ketimbang teknologi dengan paradigma *imperative*. Jadi, jika kita melihat alur waktu pengembangan sistem secara keseluruhan termasuk waktu yang dihabiskan untuk menulis kode yang sama berulang kali dan men-*debug* perulangan (*loop*) `for` yang bisa saja mengalami *off-by-one errors*, bisa saja keseluruhan waktu yang digunakan pada paradigma *imperative* lebih banyak daripada *declarative*.

Saya pribadi masih lebih suka paradigma *imperative* sewajarnya karena laptop yang saya gunakan untuk mengembangkan aplikasi Android cukup lemah dalam hal performa sehingga perdebatan antara *declarative* dan *imperative programming* kurang relevan bagi saya. Saya memang suka sintaks *declarative* yang lebih ringkas, tetapi jika ujung-ujungnya paradigma *imperative*-lah yang lebih ramah terhadap laptop saya, maka dengan senang hati saya akan menyiksa diri dengan *boilerplate* selama perangkat-perangkat saya tidak menjadi korban. Sebagaimana yang dikatakan oleh Thanos, “*Small price to pay for salvation.*”

Soal 2. Apa yang membedakan OOP pada kotlin dengan OOP pada java?

Object-oriented programming (OOP) atau pemrograman berorientasi objek (PBO) pada Kotlin tidak seketat Java.

Perbedaan yang menurut saya paling mendasar adalah pada Kotlin fungsi dapat diletakkan di luar kelas, berbeda dengan Java sebelum versi 21. Hal ini disebut *top-level function* [3]. Fungsi `main()` juga dapat diletakkan di luar kelas dan tanpa parameter, berbeda dengan Java yang mengharuskan adanya parameter `String[] args` (sekali lagi, hal ini diubah pada Java 21). Java (pra-21) mengharuskan fungsi dan variabel berada di dalam kelas karena Java menganut paham OOP yang sangat ketat.

Kemudian, di Kotlin kita dapat menambahkan fungsi baru langsung pada kelas yang sudah ada, tanpa harus membuat kelas baru yang mewarisi dari kelas itu terlebih dahulu sebagaimana di Java. Jenis fungsi seperti ini disebut *extension functions* [3]. Sekali lagi, hal ini sangat berbeda dengan paham OOP yang dianut Java (pra-21), yaitu semua fungsi harus berada di dalam kelas. Contohnya adalah `fun String.lastChar() : Char = this.get(this.length - 1)`. Di Java, kita harus membuat kelas baru terlebih dahulu, atau lebih ekstrem lagi memodifikasi kelas `String` itu sendiri, yang jelas-jelas melanggar prinsip OCP.

Kotlin juga mengharuskan kita menandai fungsi yang ditimpa dengan *modifier override*. Kalau di Java, kita bisa menggunakan anotasi `@Override`, tetapi hal itu tidak wajib sehingga bisa saja dalam tim ada seorang *programmer* yang tidak menggunakannya dan *programmer* lain tidak tahu bahwa ada fungsi yang ditimpa. Kemudian, secara *default* semua kelas dan metode di Kotlin bersifat *final*, artinya kelas atau metode tersebut tidak dapat diwarisi oleh kelas lain atau ditimpa oleh metode milik *subclass*. Kelas yang bisa ditimpa harus ditandai dengan *modifier open*. Sebaliknya, kelas yang ditandai dengan *modifier abstract* justru harus ditimpa.

Kemudian, Kotlin tidak mempunyai *package-private visibility* sebagaimana *default* di Java melalui *access modifier default*. Semuanya justru bersifat *public*.

Soal 3. Sebutkan, dan jelaskan mengenai SOLID Principles

SOLID adalah singkatan dari lima prinsip yang umumnya disingkat kembali, yaitu *single responsibility principle* (SRP), *open-closed principle* (OCP), *Liskov substitution principle* (LSP), *interface segregation principle* (ISP), dan *dependency inversion principle* (DIP). Kelima prinsip ini dikumpulkan oleh Robert C. Martin (Uncle Bob) sejak tahun 1980-an, dan pada tahun 2004 ia dikirim *e-mail* yang menyarankannya untuk menyingkat prinsip-prinsip itu menjadi SOLID [4].

Single responsibility principle (SRP) dikatakan mempunyai makna awal berupa '*a module should have one, and only one, reason to change*'. Sekarang, maknanya adalah '*A module should be responsible to one, and only one, actor*'. Maksudnya, suatu modul (baik itu berupa suatu *file* berisi *source code* maupun sekumpulan fungsi dan struktur data yang berkaitan) hanya boleh bertanggung jawab (*responsible*) kepada satu aktor, yaitu sekumpulan pengguna atau *stakeholder* [4]. Tanpa berbelit-belit, maksud sederhananya adalah suatu kelas sebaiknya hanya mempunyai fungsi yang berkaitan erat dengan satu sama lain. Suatu kelas tidak semestinya mempunyai fungsi-fungsi yang tidak saling berkaitan.

Open-closed principle (OCP) diusulkan oleh Bertrand Meyer pada tahun 1988 dan berbunyi, '*A software artifact should be open for extension but closed for modification.*' Artinya adalah *software artifact* harus dapat di-*extend* (dalam artian penambahan fungsionalitas, bukan dalam artian bisa diwarisi oleh kelas lain) tanpa harus mengubah *artifact* tersebut. Caranya adalah dengan membagi suatu sistem menjadi sejumlah komponen dan menyusun komponen tersebut menjadi hierarki *dependency* yang melindungi komponen *high-level* dari perubahan pada komponen *low-level* [4].

Liskov substitution principle (LSP) didasarkan pada pernyataan dari Barbara Liskov pada tahun yang sama dengan Bertrand Meyer mengatakan sesuatu tentang OCP di atas, yaitu 1988. Pernyataan tersebut berbunyi, '*What is wanted here is something like the following substitution property: If for each object o1 of type S there is an object o2 of type T such that for all programs P defined in terms of T, the behavior of P is unchanged when o1 is substituted for o2 then S is a subtype of T.*' [4]. Sejujurnya saya

tidak mengerti maksud pernyataan tersebut, tetapi yang saya ingat dari Pemrograman II sebelumnya adalah suatu *superclass* harus bisa digantikan oleh *subclass* tanpa merusak program. Caranya adalah dengan memastikan *subclass* mematuhi semua kriteria yang ditetapkan oleh *superclass*-nya, misalnya melalui *interface*.

Interface segregation principle (ISP) berarti operasi-operasi yang berbeda mesti dipisah menjadi *interface* yang berbeda pula, untuk mencegah kejadian seperti `User1` bergantung kepada `OPS.op1`, `User2` bergantung kepada `OPS.op2`, dan `User3` bergantung kepada `OPS.op3`, alias ketiga operasi tersebut terletak pada suatu kelas yang sama, yaitu `OPS`. Seandainya kita ingin mengubah `OPS.op2`, `User1` dan `User3` juga terpaksa menerima perubahannya meskipun yang menggunakan `op2` hanyalah `User2` [4]. ISP bisa dikatakan sebagai SRP untuk *interface*.

Dependency inversion principle (DIP) menyebutkan bahwa sistem yang fleksibel adalah sistem dengan *source code* yang hanya bergantung pada *abstraction* dan bukan *concretion*. *Abstraction* yang dimaksud mencakup hal-hal seperti *interface* dan *abstract classes* [4]. Pada dasarnya, seluruh *dependency* yang kita gunakan pada program kita sebaiknya hanyalah berupa konsep abstrak, bukan implementasi konkret karena implementasi konkret dapat berubah kapan pun. Walaupun begitu, jika suatu kelas yang kita gantungkan nasib kita itu cukup stabil seperti kelas `String` di Java, maka tidak masalah bergantung pada kelas tersebut [4].

Soal 4

Class adalah sejenis tipe data yang mirip seperti *struct* pada bahasa C++ dalam artian ia dapat menyimpan sejumlah variabel lain dengan tipe data yang berbeda di dalamnya. Kelebihan *class* dibanding *struct*, yaitu *class* dapat menyimpan fungsi juga, tidak hanya variabel. *Class* di Kotlin sendiri merupakan bentuk pengimplementasian dari paradigma pemrograman berorientasi objek (*object-oriented programming* atau OOP). *Class* dapat diinstansiasi (*instantiate*) untuk menciptakan objek pada memori. Objek itu kemudian dapat dimanipulasi sedemikian rupa, termasuk mengoperasikannya menggunakan fungsi yang tersimpan di dalamnya.

Data class di Kotlin adalah jenis *class* yang dikhususkan untuk menyimpan data saja, bukan sekaligus untuk menyimpan fungsi yang dibuat oleh pemrogram. Dalam hal ini, *data class* mirip seperti *struct*, hanya saja Kotlin *tidak* mempunyai *struct*. Tipe data yang lebih mirip dengan *data class* di Kotlin adalah *record* di Java. *Data class* secara otomatis berisi fungsi-fungsi `equals()` atau `hashCode()`, `toString()`, `componentN()` dan `copy()` tanpa mengharuskan kita membuatnya secara manual layaknya pada *class*. Pada *data class*, *primary constructor* juga harus mempunyai setidaknya satu parameter, sedangkan *class* tidak mewajibkannya. Parameter tersebut harus berupa `val` or `var`. Kemudian, *data class* tidak boleh berupa `abstract`, `open`, `sealed`, ataupun `inner` [5].

Intinya, perbedaan antara *class* dan *data class* di Kotlin terletak pada kegunaannya. Jika kita hanya ingin membuat *plain-old Java object* (POJO) atau *data transfer object* (DTO), maka gunakan *data class*. Selain itu, sebaiknya gunakan *class* biasa.

E. Daftar Pustaka

- [1] N. Nurhadi dan A. Muslim, “Comparative Analysis of the Impact of Declarative and Imperative Programming Approaches on Android Application Performance Through Benchmarking Method,” *jrssem*, vol. 5, no. 5, hlm. 5234–5248, Des 2025, doi: 10.59141/jrssem.v5i5.1231.
- [2] I. Fjodorovs dan S. Kodors, “JETPACK COMPOSE AND XML LAYOUT RENDERING PERFORMANCE COMPARISON,” *HET*, no. 25, hlm. 49–54, Apr 2021, doi: 10.17770/het2021.25.6779.
- [3] S. Aigner, R. Elizarov, S. Isakova, dan D. Jemerov, *Kotlin in action*, Second edition. Shelter Island, NY: Manning Publications Co, 2024.
- [4] R. C. Martin dan R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. dalam Robert C. Martin series. London, England: Prentice Hall, 2018.
- [5] “Data classes | Kotlin,” Kotlin Help. Diakses: 5 April 2026. [Daring]. Tersedia pada: <https://kotlinlang.org/docs/data-classes.html>

TAUTAN GIT

<https://github.com/orizynpx/praktikum-pemrograman-mobile/tree/main/Modul%201>